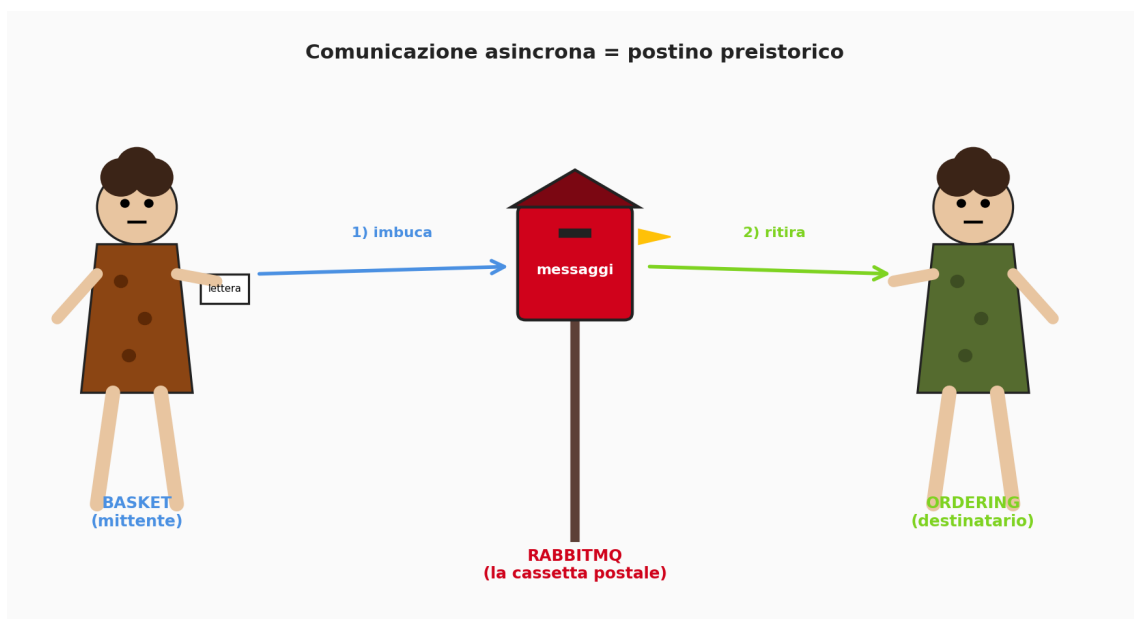


eShop

Flusso del progetto e comunicazione asincrona con il Message Broker



L'idea chiave: il Message Broker e' una cassetta postale tra due cavernicoli che non si parlano direttamente.

Documento tecnico

Architettura microservizi .NET 8 - RabbitMQ - MassTransit

Indice

PARTE I - LA MESSAGGISTICA ASINCRONA

- 1. Panoramica del progetto eShop..... 3
- 2. Architettura generale..... 4
- 3. Le tre forme di comunicazione..... 6
- 4. Il broker spiegato all'uomo delle caverne..... 7
- 5. I tre personaggi: Publisher, Broker, Consumer..... 9
- 6. MassTransit: chi e' e cosa fa..... 11
- 7. Il flusso completo del checkout (passo passo)..... 13
- 8. Cosa accade DENTRO RabbitMQ..... 16
- 9. Cosa fa MassTransit dietro le quinte (8 step)..... 17

PARTE II - LA PIPELINE HTTP DEL PROGETTO

- 10. Carter: il routing modulare..... 19
- 11. Mapster: il mapping invisibile..... 21
- 12. La pipeline completa di una richiesta HTTP..... 22

PARTE III - I DESIGN PATTERN USATI

- 13. Panoramica dei pattern..... 24
- 14. CQRS Pattern..... 25
- 15. Mediator Pattern..... 26
- 16. Pipeline / Chain of Responsibility..... 27
- 17. Decorator + Proxy..... 28
- 18. Repository Pattern..... 30
- 19. Aggregate Root, Factory Method, Domain Events (DDD)..... 31
- 20. Interceptor Pattern (EF Core)..... 33
- 21. Riepilogo e punti chiave da ricordare..... 34

1. Panoramica del progetto eShop

Il progetto **eShop** e' un e-commerce sviluppato in **.NET 8** seguendo lo stile architetturale a **microservizi**. Invece di una singola applicazione monolitica, ogni dominio funzionale (catalogo prodotti, carrello, ordini, sconti) vive in un servizio autonomo con il proprio database e il proprio ciclo di vita.

I quattro microservizi

Catalog API (porta 6000): gestisce i prodotti del catalogo. Persistenza su PostgreSQL tramite Marten.

Basket API (porta 6001): gestisce il carrello dell'utente. Persistenza su PostgreSQL e cache su Redis. E' anche il publisher dei nostri eventi di checkout.

Discount gRPC (porta 6002): servizio interno per il calcolo degli sconti, esposto via gRPC. Persistenza su SQLite.

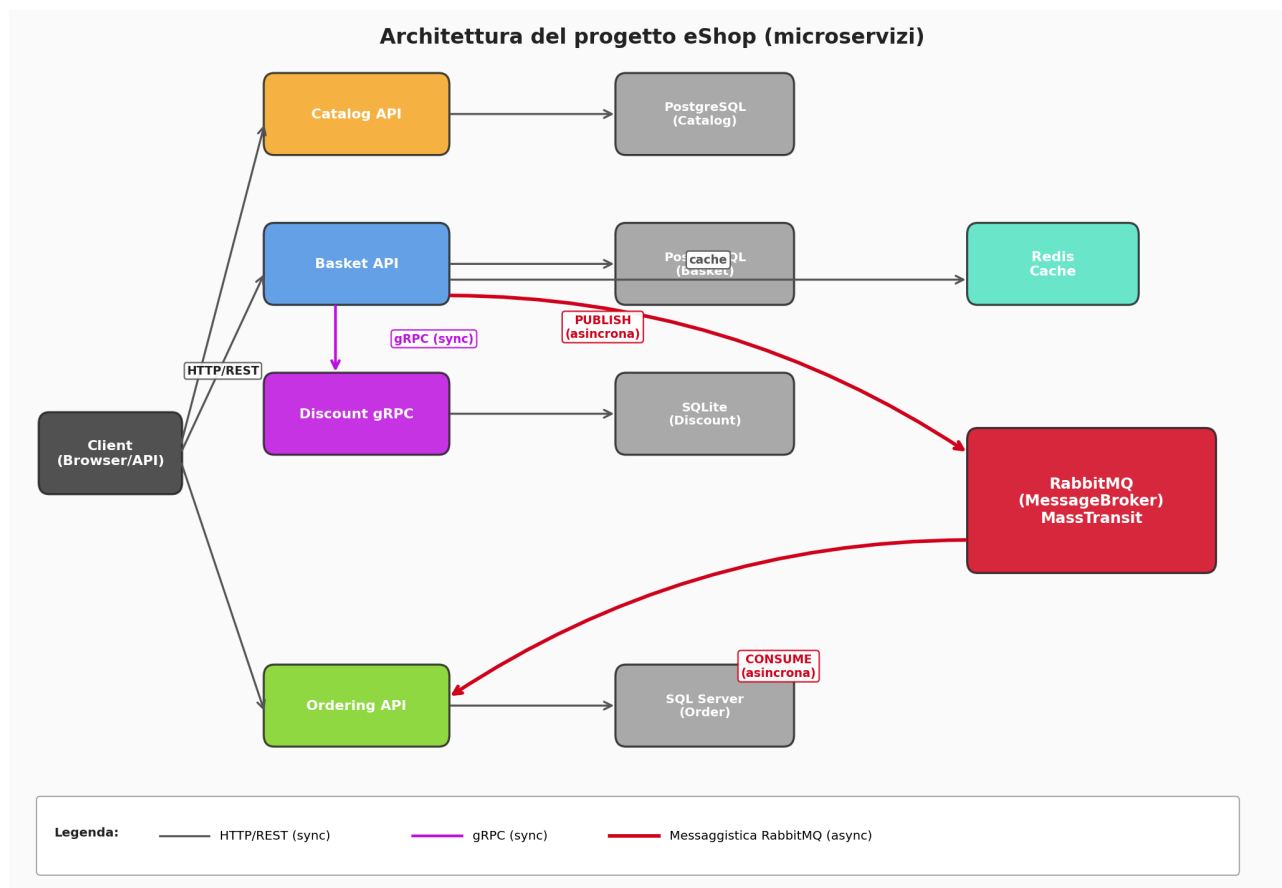
Ordering API: gestisce gli ordini. Persistenza su SQL Server, segue i pattern di **Domain-Driven Design** (aggregati, value objects, domain events) e **CQRS**. E' il consumer degli eventi di checkout.

Stack tecnologico in breve

- .NET 8, ASP.NET Core Minimal API, Carter (per gli endpoint)
- MediatR per il pattern CQRS dentro ogni servizio
- Marten + PostgreSQL (Catalog/Basket), EF Core + SQL Server (Ordering)
- Redis come distributed cache nel Basket
- gRPC per la chiamata sincrona Basket -> Discount
- **RabbitMQ** + **MassTransit** per la comunicazione asincrona Basket -> Ordering
- Docker Compose per orchestrare tutti i container

2. Architettura generale

Lo schema seguente mostra **chi parla con chi** e **come**. Le frecce nere sono chiamate HTTP/REST classiche, quelle viola sono chiamate **gRPC** (sincrone), e quelle **rosse** rappresentano i messaggi che passano dal Message Broker (asincroni).



Vista d'insieme: client, microservizi, database e RabbitMQ.

Ogni microservizio ha il **proprio database** - questo è un principio fondamentale dell'architettura a microservizi: nessuno scrive nel DB di un altro servizio. Se Ordering ha bisogno di sapere che è avvenuto un checkout, non legge dal DB del Basket: aspetta che Basket gli mandi un **messaggio**.

Perche' tante tecnologie diverse?

Ogni servizio sceglie lo strumento più adatto al proprio dominio. Il Catalog ha pochi modelli e ama PostgreSQL+Marten; Ordering ha relazioni complesse e regole di business ricche, quindi sceglie EF Core+DDD; il Basket sfrutta Redis perché i carrelli sono dati hot a vita breve. Questa eterogeneità è il prezzo (e il superpotere) dei microservizi.

3. Le tre forme di comunicazione

Nel progetto convivono tre modi diversi di far parlare due servizi. Capirne la differenza e' la chiave per capire *perche'* il Message Broker esiste.

a) HTTP/REST (sincrona)

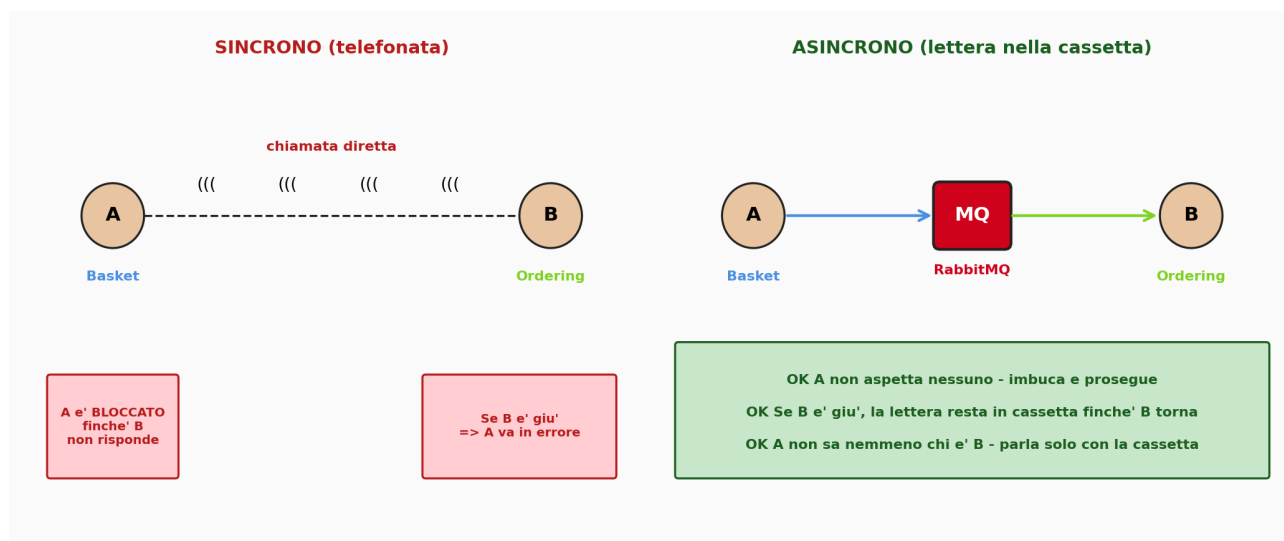
Il client (browser, Postman, ecc.) chiama via HTTP gli endpoint dei microservizi - per esempio POST /basket/checkout. E' una richiesta-risposta classica: aspetto la risposta prima di proseguire.

b) gRPC (sincrona, tra microservizi)

Quando il **Basket** ha bisogno di calcolare lo sconto su un prodotto, chiama il microservizio **Discount** via gRPC. E' anche questa una chiamata *sincrona*: Basket aspetta la risposta del servizio Discount prima di andare avanti. gRPC e' piu' veloce di HTTP/JSON perche' usa Protocol Buffers e HTTP/2, ma il principio resta lo stesso: *se Discount e' giu', Basket fallisce*.

c) Messaggistica via Broker (asincrona)

Quando il **Basket** deve dire a **Ordering** che l'utente ha completato il checkout, **non** lo chiama direttamente. Invece **pubblica un evento** su RabbitMQ. Ordering, quando puo', leggerà quell'evento e creerà l'ordine. Il Basket non sa nemmeno se Ordering esista: parla solo con la cassetta postale.



Sincrono = telefonata. Asincrono = lettera nella cassetta.

4. Il broker spiegato all'uomo delle caverne

Da qui in poi rallentiamo. Niente termini tecnici per qualche pagina: racconto la stessa cosa come se stessi parlando a un cavernicolo. Poi torniamo nei servizi.

■ L'analogia base: la cassetta postale del villaggio

Immagina due cavernicoli, **Ugo** e **Bork**, che vivono lontani. Ugo fa lance, Bork le usa per cacciare.

Ugo NON ha tempo di andare ogni giorno fino alla grotta di Bork per chiedergli se gli serve una lancia nuova. E Bork potrebbe non essere in casa, magari sta cacciando.

Allora gli abitanti del villaggio mettono in mezzo una **cassetta postale**. Quando Ugo finisce una lancia, scrive un foglietto - '*Lancia nuova pronta*' - e lo imbuca. Stop. Ugo torna a fare lance.

Bork, quando torna, passa davanti alla cassetta, prende i foglietti e fa quello che ci sta scritto.

Ugo non sa quando Bork passerà. Non gli interessa: ha consegnato il messaggio alla cassetta e ha fatto la sua parte.

■ Perché la cassetta postale è meglio del 'urlare a Bork'?

- 1) Ugo non perde tempo aspettando: imbuca e prosegue il suo lavoro.
- 2) Se Bork è malato e sta una settimana in grotta, le lettere si accumulano nella cassetta. Niente va perso.
- 3) Se in futuro arriva un terzo cavernicolo (Grug, che fa cesti) anche lui potrà iscriversi alla cassetta e leggere lo stesso foglietto. Ugo non deve cambiare nulla.
- 4) Se Ugo e Bork litigano, possono comunicare comunque: parlano alla cassetta, mai tra loro.

■ **Tradotto in termini moderni:**

- Ugo (mittente) = **Basket.API** (il servizio che pubblica)
- Bork (destinatario) = **Ordering.API** (il servizio che consuma)
- La cassetta postale = **RabbitMQ** (il Message Broker)
- Il foglietto = il **messaggio/evento** (BasketCheckoutEvent)
- Il fattore con il timbro = **MassTransit** (la libreria che gestisce le buste)

5. I tre personaggi: Publisher, Broker, Consumer

Ora che abbiamo l'immagine in testa, mettiamoci sopra i nomi tecnici. In ogni sistema basato su messaggi ci sono **sempre tre ruoli**:



I tre ruoli fondamentali nella messaggistica asincrona.

Publisher = Basket.API

È il servizio che **produce e pubblica** un evento. Nel nostro progetto il Basket pubblica un evento di tipo `BasketCheckoutEvent` ogni volta che un utente conferma il checkout.

Il publisher non sa, e non deve sapere, chi consumerà il messaggio.

Broker = RabbitMQ

È un programma a sé che gira come container Docker (vedi `docker-compose.yml`). Riceve messaggi da chiunque, li conserva in code (queue) e li consegna ai consumatori interessati. Mantiene i messaggi in memoria **e su disco** finché qualcuno non li legge (e finché non si conferma la lettura).

Consumer = Ordering.API

È il servizio che **ascolta** il broker. Nel nostro progetto il file `BasketCheckoutEventHandler` è il consumer: viene risvegliato ogni volta che arriva un messaggio dal broker e fa qualcosa con esso (creare un ordine).

i Definizione: Integration Event

Nel codice del progetto esiste una classe base `IntegrationEvent` (in `BuildingBlocks.Messaging.Events`). Si chiama *integration* perche' viaggia tra servizi diversi - integra il sistema. `BasketCheckoutEvent` eredita da questa classe.

6. MassTransit: chi e' e cosa fa

RabbitMQ da solo parla un linguaggio chiamato **AMQP**. Programmare direttamente con AMQP significa scrivere codice di basso livello: aprire connessioni, dichiarare exchange, dichiarare queue, configurare il binding, serializzare bit a bit. **MassTransit** e' una libreria .NET che fa tutto questo lavoro al posto tuo. Tu lavori con classi C#, lui parla con RabbitMQ.

Setup nel progetto: BuildingBlocks.Messaging

Il modulo condiviso BuildingBlocks.Messaging contiene un metodo di estensione AddMessageBroker che entrambi i servizi (Basket e Ordering) usano per registrare MassTransit + RabbitMQ:

```
public static IServiceCollection AddMessageBroker(
    this IServiceCollection services,
    IConfiguration configuration,
    Assembly? assembly = null)
{
    services.AddMassTransit(cfg =>
    {
        cfg.SetKebabCaseEndpointNameFormatter();
        if (assembly is not null)
            cfg.AddConsumers(assembly); // registra TUTTI gli IConsumer<T>

        cfg.UsingRabbitMq((context, configurator) =>
        {
            configurator.Host(new Uri(configuration["MessageBroker:Host"]), h =>
            {
                h.Username(configuration["MessageBroker:Username"]);
                h.Password(configuration["MessageBroker:Password"]);
            });
            configurator.ConfigureEndpoints(context);
        });
    });
    return services;
}
```

Cosa succede in queste righe:

- **SetKebabCaseEndpointNameFormatter**: il nome dell'evento in C# (BasketCheckoutEvent) viene tradotto in basket-checkout-event per i nomi delle queue.
- **AddConsumers(assembly)**: scansiona l'assembly e registra automaticamente tutte le classi che implementano IConsumer<T>. Ordering passa l'assembly di Ordering.Application perche' li' vive il BasketCheckoutEventHandler.
- **UsingRabbitMq(...)**: dice a MassTransit di usare RabbitMQ come trasporto, e configura host/credenziali leggendo da appsettings.json.
- **ConfigureEndpoints(context)**: crea automaticamente le queue per ciascun consumer trovato nel passo precedente.

Configurazione (appsettings.json)

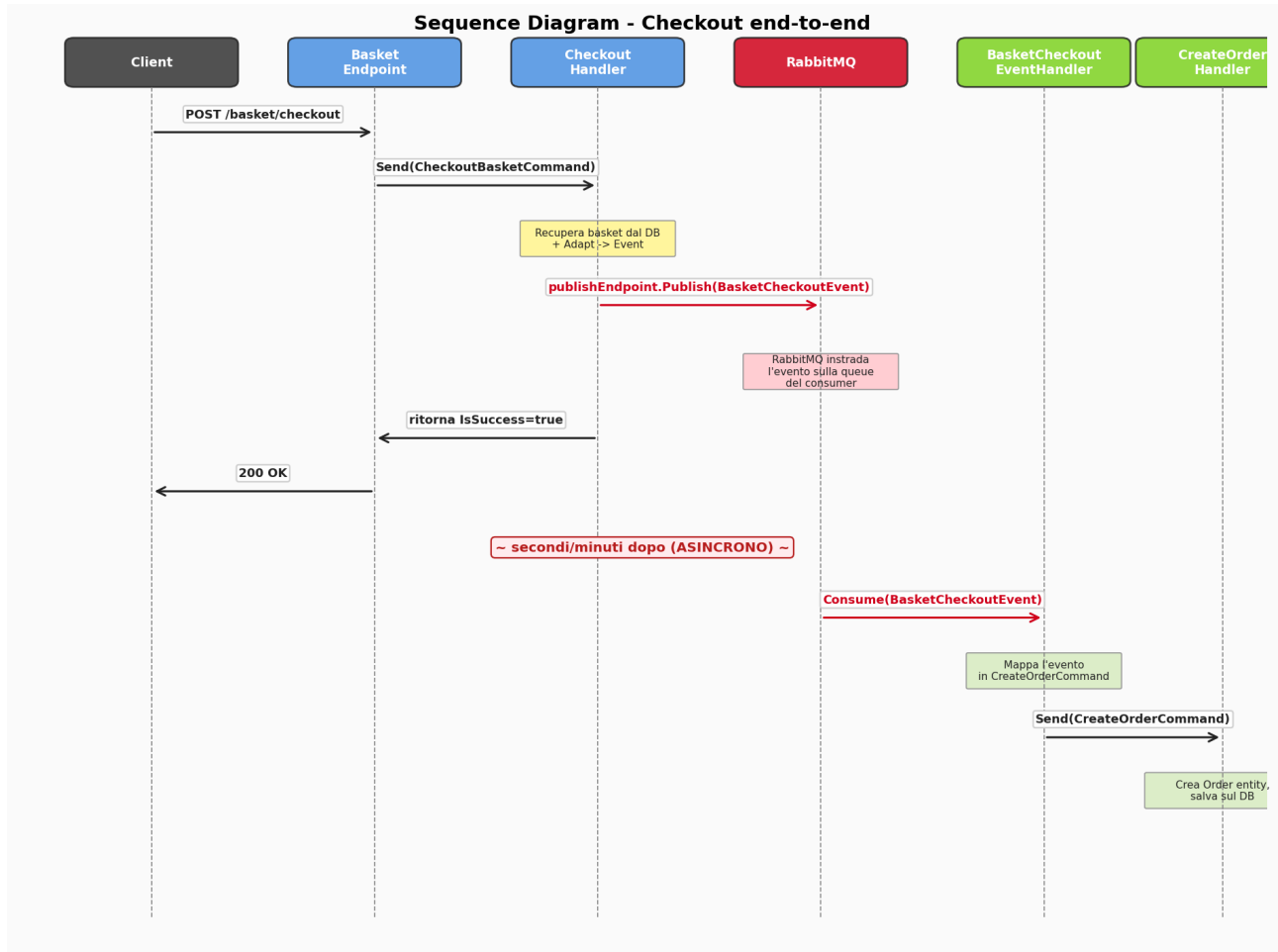
Sia Basket sia Ordering hanno questa sezione:

```
"MessageBroker": {  
  "Host": "amqp://localhost:5672",  
  "Username": "guest",  
  "Password": "guest"  
}
```

amqp://... e' l'URL del protocollo che parla con RabbitMQ. In Docker il host diventa messagebroker (il nome del container).

7. Il flusso completo del checkout (passo passo)

Mettiamo insieme tutti i pezzi. Lo scenario e': l'utente preme 'Conferma ordine' nel carrello. Vediamo cosa succede a livello di ogni servizio.



Sequence diagram: il giro del checkout dal click al record sul DB.

Step 1: Il client chiama l'endpoint Basket

Il client fa una `POST /basket/checkout` con i dati del carrello e di pagamento. L'endpoint vive in `CheckoutBasketEndpoints`:

```

app.MapPost("/basket/checkout", async (CheckoutBasketRequest request, ISender sender) =>
{
    var command = request.Adapt<CheckoutBasketCommand>();
    var result = await sender.Send(command); // -> CheckoutBasketCommandHandler
    var response = result.Adapt<CheckoutBasketResponse>();
    return Results.Ok(response);
});
  
```

Step 2: L'handler del Basket pubblica l'evento

Il vero cuore del publisher e' `CheckoutBasketCommandHandler`:

```
public class CheckoutBasketCommandHandler(
    IBasketRepository basketRepository,
    IPublishEndpoint publishEndpoint) // <-- iniettato da MassTransit
    : ICommandHandler<CheckoutBasketCommand, CheckoutBasketResult>
{
    public async Task<CheckoutBasketResult> Handle(
        CheckoutBasketCommand command, CancellationToken ct)
    {
        // 1. Recupera il basket
        var basket = await basketRepository.GetBasketAsync(
            command.BasketCheckoutDto.UserName, ct);
        if (basket is null) return new CheckoutBasketResult(false);

        // 2. Costruisce l'evento di integrazione
        var eventMessage = command.BasketCheckoutDto.Adapt<BasketCheckoutEvent>();
        eventMessage.TotalPrice = basket.TotalPrice;

        // 3. Lo PUBBLICA sul broker
        await publishEndpoint.Publish(eventMessage, ct);

        // 4. Cancella il basket dal DB
        await basketRepository.DeleteBasketAsync(
            command.BasketCheckoutDto.UserName, ct);

        return new CheckoutBasketResult(true);
    }
}
```

Tre cose da notare: **(1)** il publisher non chiama nessuna API di Ordering; **(2)** riceve un'astrazione (`IPublishEndpoint`) e non sa nulla di RabbitMQ; **(3)** finita la chiamata a `Publish`, ritorna subito al client - non aspetta che l'ordine venga creato.

Step 3: RabbitMQ riceve e instrada

MassTransit serializza l'oggetto C# in JSON e lo manda a RabbitMQ. RabbitMQ lo conserva nella coda `basket-checkout-event` (creata automaticamente). A questo punto il client ha già ricevuto un `200 OK`: il checkout, dal suo punto di vista, è concluso.

Step 4: Ordering consuma il messaggio

Il `BasketCheckoutEventHandler` in `Ordering.Application/Orders/EventHandlers/Integration` implementa `IConsumer<BasketCheckoutEvent>`. MassTransit lo invoca quando arriva un messaggio sulla queue:

```
public class BasketCheckoutEventHandler(ISender sender, ILogger<...> logger)
    : IConsumer<BasketCheckoutEvent>
{
    public async Task Consume(ConsumeContext<BasketCheckoutEvent> context)
    {
        logger.LogInformation("Integration Event Handled: {Type}",
            context.Message.GetType().Name);

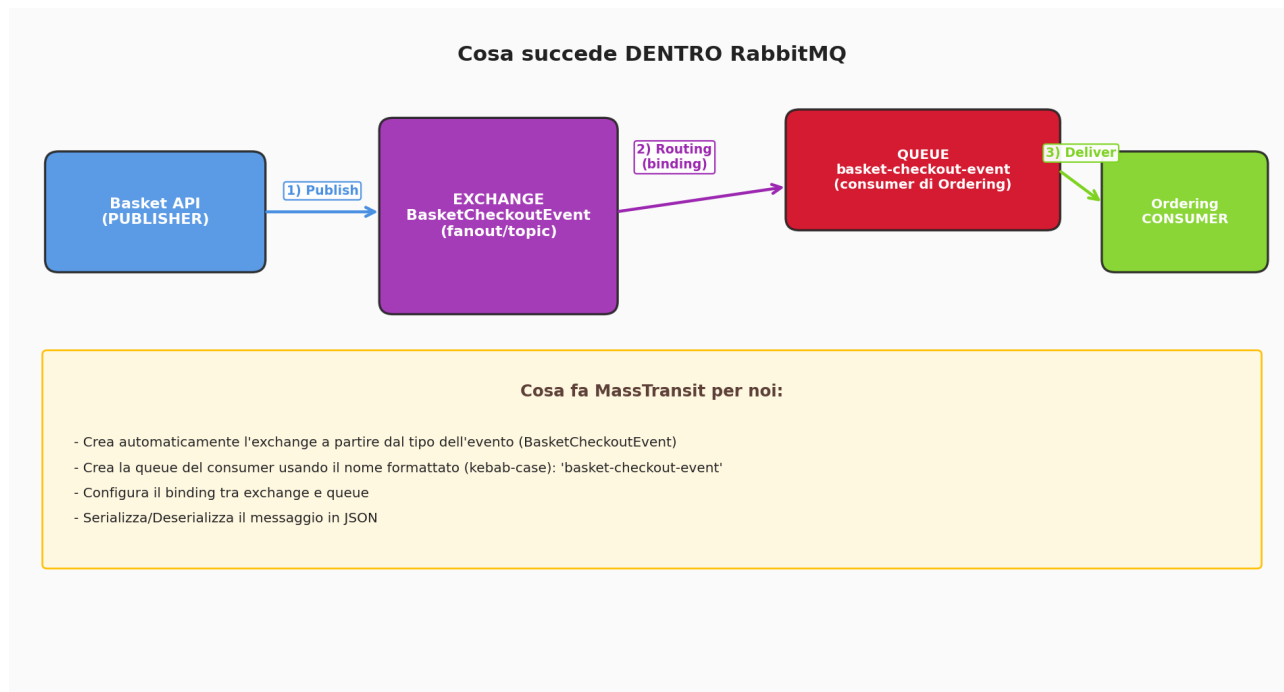
        // 1) Trasforma l'evento in un comando di dominio
        var command = MapToCreateOrderCommand(context.Message);

        // 2) Lo invia al CreateOrderHandler tramite MediatR
        await sender.Send(command);
    }
}
```

Da qui in poi siamo nel mondo Ordering: il `CreateOrderHandler` crea l'aggregato `Order` e lo persiste su SQL Server.

8. Cosa accade DENTRO RabbitMQ

Per chi vuole sapere cosa succede al messaggio dopo il Publish, RabbitMQ usa tre concetti chiave: **exchange**, **queue**, **binding**.



Exchange + Queue + Binding: il triangolo magico di RabbitMQ.

- **Exchange:** il punto in cui i messaggi *entrano*. Pensalo come lo sportello d'ingresso della cassetta. Non conserva nulla - smista soltanto.
- **Queue:** la vera 'coda' dove il messaggio aspetta finché qualcuno non lo legge. Persistita su disco se serve.
- **Binding:** la regola che dice *'i messaggi che entrano in questo exchange devono finire in questa queue'*. E' la cinghia di trasmissione.

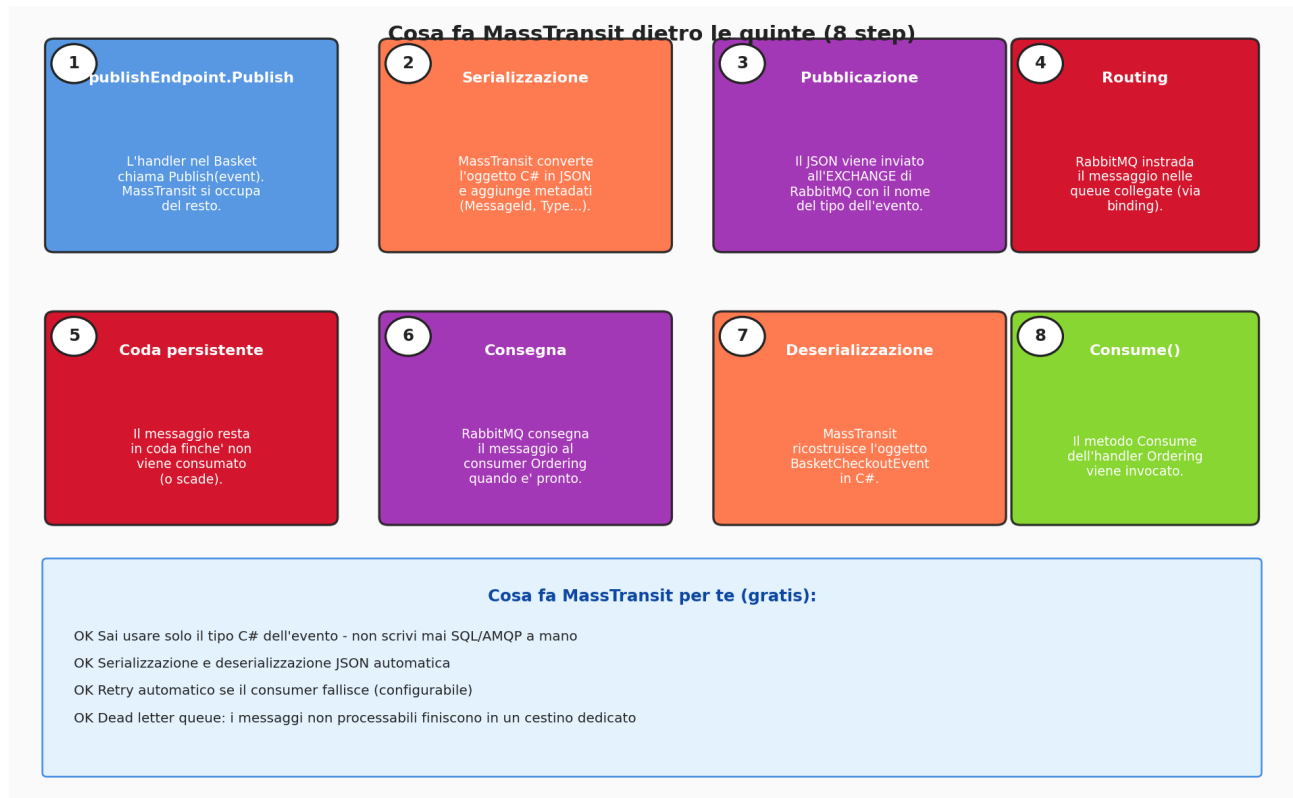
MassTransit, grazie a `SetKebabCaseEndpointNameFormatter`, crea automaticamente: un exchange chiamato come il tipo (`BasketCheckoutEvent`), una queue chiamata come il consumer (`basket-checkout-event`), e il binding che li unisce. Non scriviamo una sola riga di AMQP.

* Vuoi vedere tutto questo dal vivo?

RabbitMQ esposto sulla porta 15672 ha una **UI di management**. Apri `http://localhost:15672` (user/pass: guest/guest) e troverai tab dedicati a Exchanges, Queues, Connections. Pubblica un checkout e guarda il messaggio scorrere in tempo reale.

9. Cosa fa MassTransit dietro le quinte (8 step)

Per chiudere, ecco cosa accade dal momento in cui chiami Publish al momento in cui il tuo Consume viene invocato:



La pipeline completa: 8 step che MassTransit gestisce per te.

Step 1-4 avvengono nel processo del Basket; **step 5** vive in RabbitMQ; **step 6-8** avvengono nel processo dell'Ordering. Tu, come sviluppatore, scrivi solo lo step 1 (Publish) e lo step 8 (Consume). Tutto il resto è meccanica del framework.

+ Bonus che ottieni gratis (configurabili)

Retry: se Consume lancia un'eccezione, MassTransit ritenta automaticamente N volte prima di arrendersi.

Dead Letter Queue: i messaggi che continuano a fallire vengono spostati in una queue speciale (error) così non bloccano il flusso principale e tu puoi ispezionarli con calma.

Idempotenza: ogni messaggio ha un MessageId unico - se lo vedi due volte sai che è duplicato.

Tracing: integrazione automatica con OpenTelemetry, log strutturati e metriche.

10. Carter: il routing modulare

Nel progetto NON ci sono Controller classici ([ApiController]). Al loro posto trovi una libreria che si chiama **Carter** e una serie di classi che implementano ICarterModule. Ogni modulo Carter espone un piccolo gruppo di endpoint via Minimal API.

Cos'e' Carter

Carter e' una libreria leggera che organizza le Minimal API di ASP.NET in **moduli auto-contenuti**. Invece di scrivere tutti gli endpoint dentro Program.cs, ne crei uno per feature. Si registra così:

```
// in Program.cs
builder.Services.AddCarter();
...
app.MapCarter(); // scopre tutti gli ICarterModule e li mappa
```

Esempio dal Catalog

Ecco un modulo Carter completo (CreateProductEndpoint):

```
public record CreateProductRequest(
    string Name, List<string> Categories, string Description,
    string ImageFile, decimal Price);

public record CreateProductResponse(Guid Id);

public class CreateProductEndpoint : ICarterModule
{
    public void AddRoutes(IEndpointRouteBuilder app)
    {
        app.MapPost("/products",
            async (CreateProductRequest request, ISender sender) =>
            {
                var command = request.Adapt<CreateProductCommand>(); // Mapster
                var result = await sender.Send(command); // MediatR
                var response = result.Adapt<CreateProductResponse>(); // Mapster

                return Results.Created($" /products/{response.Id}", response);
            })
            .WithName("CreateProduct")
            .Produces<CreateProductResponse>(StatusCodes.Status201Created)
            .ProducesProblem(StatusCodes.Status400BadRequest);
    }
}
```

Cosa guadagni con Carter

- **Modularita'**: ogni feature ha la sua classe di endpoint - facile trovare e modificare.
- **Pulizia**: niente Controller bloated; il modulo si occupa solo di routing + glue, la logica vive nei Command/Handler.
- **Auto-discovery**: app.MapCarter() trova tutti gli ICarterModule via riflessione - non devi registrarli a mano.

- **OpenAPI/Swagger** via `.WithName()`, `.Produces<T>()`, `.ProducesProblem(...)` - documentazione co-localizzata con la rotta.

Carter come Module Pattern

A livello di design pattern, Carter e' un'implementazione del classico **Module Pattern**: ogni modulo dichiara cio' che espone e si auto-registra. E' la stessa filosofia di moduli AngularJS, NestJS, ecc. - solo applicata alle rotte HTTP.

Nel progetto eShop puoi vedere che ogni cartella feature ha il suo modulo:

```
Catalog/CatalogAPI/  
  Products/  
    CreateProduct/  -> CreateProductEndpoint  (ICarterModule)  
    GetProducts/    -> GetProductsEndpoint    (ICarterModule)  
    UpdateProduct/  -> UpdateProductEndpoint  (ICarterModule)  
    DeleteProduct/  -> DeleteProductEndpoint  (ICarterModule)
```

i Vertical Slice Architecture

Ogni feature contiene insieme: **Endpoint (Carter)**, **Command/Query (MediatR)**, **Handler**, **Validator**. Si chiama *Vertical Slice Architecture* - tagli verticali in cui ogni 'fetta' contiene tutto il necessario per quella feature. Il vantaggio: cambiando una feature, modifichi solo quella cartella.

11. Mapster: il mapping invisibile

Mapster e' una libreria di mapping tra oggetti, alternativa performante ad AutoMapper. Nel progetto compare ovunque tu veda il metodo `.Adapt<T>()`.

A cosa serve

Quando arriva una richiesta HTTP, hai un **Request DTO**. Quando deleghi al MediatR, ti serve un **Command**. Quando ricevi il **CommandResult**, lo devi convertire in **Response DTO**. Ogni passaggio richiederebbe codice come:

```
// senza Mapster (verbose)
var command = new CreateProductCommand(
    Name:      request.Name,
    Categories: request.Categories,
    Description: request.Description,
    ImageFile:  request.ImageFile,
    Price:      request.Price
);
```

Con Mapster diventa:

```
// con Mapster (elegante)
var command = request.Adapt<CreateProductCommand>();
```

Mapster guarda le proprietà di request e quelle di CreateProductCommand e copia i campi **omonimi**. Funziona out-of-the-box quando i campi corrispondono - per i casi non banali supporta configurazioni custom.

Mapster nel CheckoutBasket

Lo stesso meccanismo trasforma il DTO di checkout nell'Integration Event che viaggia su RabbitMQ:

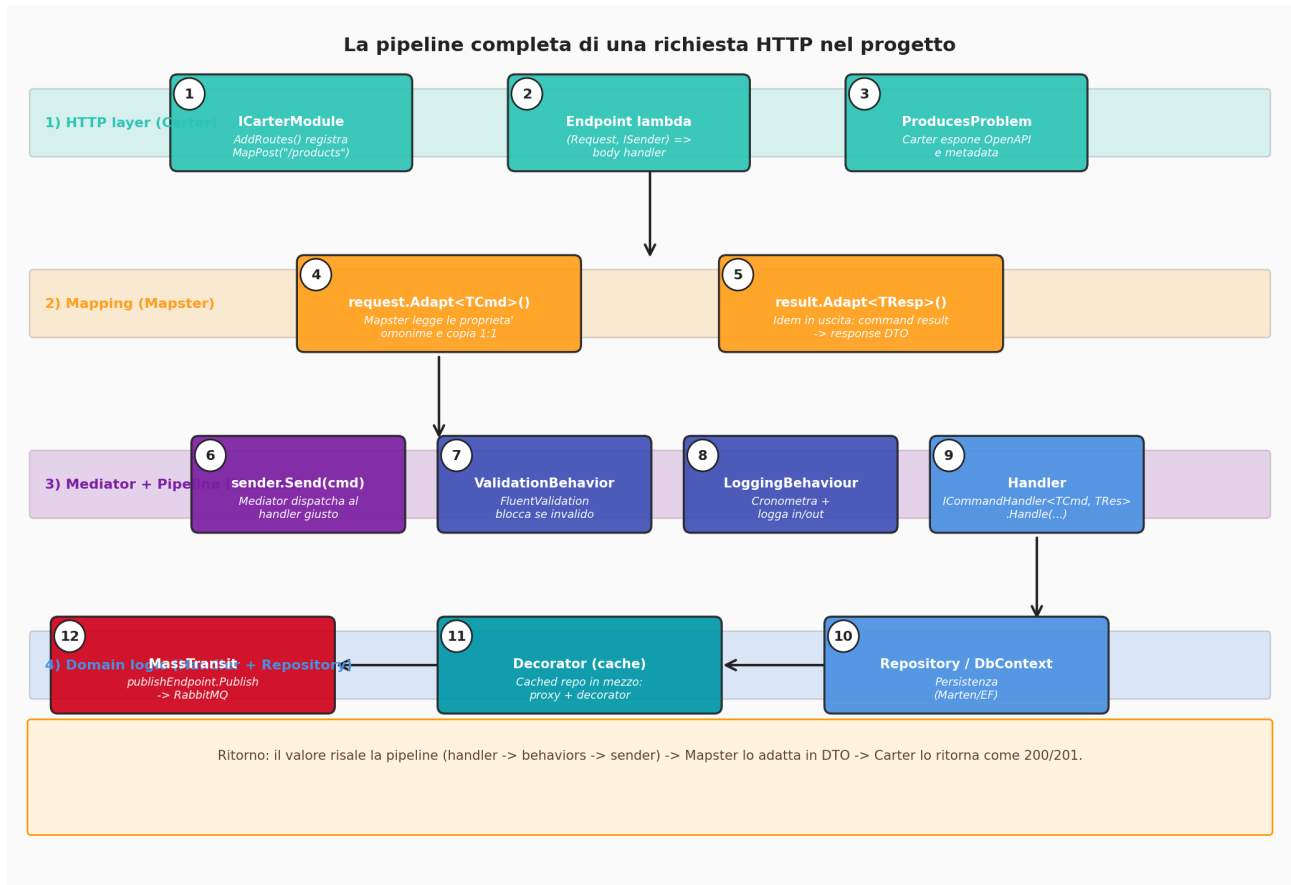
```
// dentro CheckoutBasketCommandHandler
var eventMessage = command.BasketCheckoutDto.Adapt<BasketCheckoutEvent>();
eventMessage.TotalPrice = basket.TotalPrice;
await publishEndpoint.Publish(eventMessage, ct);
```

i Pattern: Adapter di tipo runtime

Mapster e' una concretizzazione del classico **Adapter Pattern**: fa parlare due tipi che hanno la stessa forma ma classi diverse, senza bisogno di scrivere codice di mapping a mano. La differenza con un 'classico' Adapter e' che la mappatura e' decisa a runtime via riflessione (o a compile-time, se usi il source generator di Mapster).

12. La pipeline completa di una richiesta HTTP

Mettiamo insieme Carter, Mapster e MediatR per vedere il **viaggio completo** di una request, dal momento in cui arriva al server al momento in cui torna indietro la response. Questa e' la stessa pipeline che ogni feature del progetto attraversa - capisci una, capisci tutte.



Le 12 fasi che attraversa una richiesta HTTP nel progetto.

I quattro **livelli** orizzontali corrispondono a quattro responsabilita' diverse:

- **HTTP layer (Carter):** parla solo di rotte, status code e DTO. Non sa nulla di logica di business.
- **Mapping layer (Mapster):** sposta dati tra DTO e command/result. Una linea di codice, niente boilerplate.
- **Mediator layer (MediatR):** dispatcha il command al suo handler e applica i *pipeline behaviors* (validazione, logging, ...) prima di chiamare il vero handler.
- **Domain layer:** l'handler esegue la logica vera, parla con repository e/o pubblica eventi su RabbitMQ.

Punto chiave: la response risale la pipeline al contrario. Mapster trasforma il `CommandResult` in `Response DTO`, e Carter lo rende come 200 OK o 201 Created.

13. Panoramica dei design pattern usati

Nel progetto eShop sono stati applicati **una dozzina di design pattern**, molti dei quali co-esistono nello stesso flusso. Questa sezione li elenca tutti; nei capitoli successivi ne analizziamo i più importanti uno per uno.



I 12 pattern adottati - alcuni espliciti, altri impliciti ma riconoscibili nel codice.

Pattern non analizzati in dettaglio nei capitoli seguenti, ma comunque presenti nel progetto:

- **Module Pattern** (gia' visto): ICarterModule e' la sua implementazione.
- **Adapter Pattern** (gia' visto): Mapper Io concretizza.
- **Pub/Sub Pattern** (gia' visto in PARTE I): BasketCheckoutEvent via RabbitMQ.
- **Value Object** (DDD): OrderId, Address, Payment, OrderName - oggetti immutabili identificati dal valore, non da un id.

14. CQRS Pattern

CQRS = *Command Query Responsibility Segregation*. Il principio e' semplice: **separare la lettura dalla scrittura**. Le operazioni che modificano lo stato sono **Command**; quelle che leggono soltanto sono **Query**. I due percorsi non si mescolano.

Implementazione nel progetto

In BuildingBlocks/CQRS trovi quattro interfacce molto piccole:

```
// Comandi (modificano stato)
public interface ICommand : IRequest<Unit> { }
public interface ICommand<out TResponse> : IRequest<TResponse> { }

public interface ICommandHandler<in TCommand, TResponse>
    : IRequestHandler<TCommand, TResponse>
    where TCommand : ICommand<TResponse>
    where TResponse : notnull
{
}

// Query (sola lettura)
public interface IQuery<out TResponse> : IRequest<TResponse>
    where TResponse : notnull { }

public interface IQueryHandler<in TQuery, TResponse>
    : IRequestHandler<TQuery, TResponse>
    where TQuery : IQuery<TResponse>
    where TResponse : notnull { }
```

Sono **marker interfaces**: estendono IRequest di MediatR ma con la sola finalita' di chiarire l'intento. Il compilatore impedisce di confondere un ICommand con un IQuery.

Esempi nel codice

- CreateProductCommand (Catalog), CheckoutBasketCommand (Basket), CreateOrderCommand (Ordering) implementano ICommand<TResult>.
- GetOrdersQuery, GetOrdersByCustomerQuery, GetProductsQuery implementano IQuery<TResult>.

i Perche' CQRS conviene

Chiarezza: chi guarda il codice capisce subito se l'operazione modifica o no.

Test piu' semplici: i Command Handler si testano con un fake repository; le Query Handler con dati read-only.

Possibilita' di scalare in futuro: in scenari estremi, le query possono leggere da un read-model dedicato (DB diverso, materializzato), mentre i command continuano a scrivere sul DB principale.

15. Mediator Pattern

Il **Mediator** e' un oggetto che riceve *messaggi* (command/query/event) da chi li produce e li smista a chi sa gestirli, senza che produttore e consumatore si conoscano direttamente.

MediatR nel progetto

La libreria **MediatR** (notare il capolettera diverso da MassTransit) implementa il pattern. Si usa cosi':

```
// Setup (Program.cs)
builder.Services.AddMediatR(cfg =>
{
    cfg.RegisterServicesFromAssembly(assembly);
    cfg.AddOpenBehavior(typeof(ValidationBehavior<,>));
    cfg.AddOpenBehavior(typeof(LoggingBehaviour<,>));
});

// Uso (in qualunque endpoint)
public async Task<IResult> Handle(MyRequest req, ISender sender)
{
    var command = req.Adapt<MyCommand>();
    var result = await sender.Send(command); // <-- mediator
    return Results.Ok(result);
}
```

Cosa risolve il Mediator

- **Disaccoppia mittente e destinatario:** l'endpoint NON sa quale classe gestisce il comando. Sa solo il nome del comando.
- **Niente DI a mano:** non devi iniettare dieci handler diversi in ogni Controller. Inietti un solo ISender.
- **Apri la strada al Pipeline pattern:** tra il Send e l'handler reale possono inserirsi behaviors trasversali (validazione, logging, transazioni).

! Mediator vs Message Broker - non confondere

MediatR e' *in-process*: il mediator vive nello stesso processo del mittente; le chiamate sono sincrone (await).

RabbitMQ + MassTransit e' *out-of-process*: i mittenti e i destinatari sono in processi diversi, anche su macchine diverse, e si scambiano messaggi attraverso la rete.

Entrambi seguono lo stesso schema concettuale (mittente -> mediator -> destinatario), ma vivono a livelli architetturali diversi.

16. Pipeline / Chain of Responsibility

Tra il momento in cui chiami `sender.Send(command)` e il momento in cui il vero handler riceve il `command`, MediatR applica una **catena di behaviors** registrati. Ogni behavior puo': eseguire codice prima, decidere se chiamare `next()`, eseguire codice dopo. E' il classico **Chain of Responsibility** - ASP.NET Middleware funziona esattamente cosi'.

ValidationBehavior - blocca se non valido

```
public class ValidationBehavior<TRequest, TResponse>
    (IEnumerable<IValidator<TRequest>> validators)
    : IPipelineBehavior<TRequest, TResponse>
    where TRequest : ICommand<TResponse>
{
    public async Task<TResponse> Handle(
        TRequest request,
        RequestHandlerDelegate<TResponse> next,
        CancellationToken cancellationToken)
    {
        var ctx = new ValidationContext<TRequest>(request);
        var validationResults = await Task.WhenAll(
            validators.Select(v => v.ValidateAsync(ctx, cancellationToken)));
        var failures = validationResults
            .Where(x => x.Errors.Any())
            .SelectMany(r => r.Errors).ToList();

        if (failures.Any())
            throw new ValidationException(failures);

        return await next(); // passa al prossimo behavior / handler
    }
}
```

LoggingBehaviour - cronometra e logga

```
public class LoggingBehaviour<TRequest, TResponse>(...)
    : IPipelineBehavior<TRequest, TResponse>
    where TRequest : notnull, IRequest<TResponse>
    where TResponse : notnull
{
    public async Task<TResponse> Handle(TRequest request,
        RequestHandlerDelegate<TResponse> next, CancellationToken ct)
    {
        logger.LogInformation($"[START] Handler {typeof(TRequest).Name}");
        var timer = Stopwatch.StartNew();

        var response = await next(); // chiama il prossimo anello

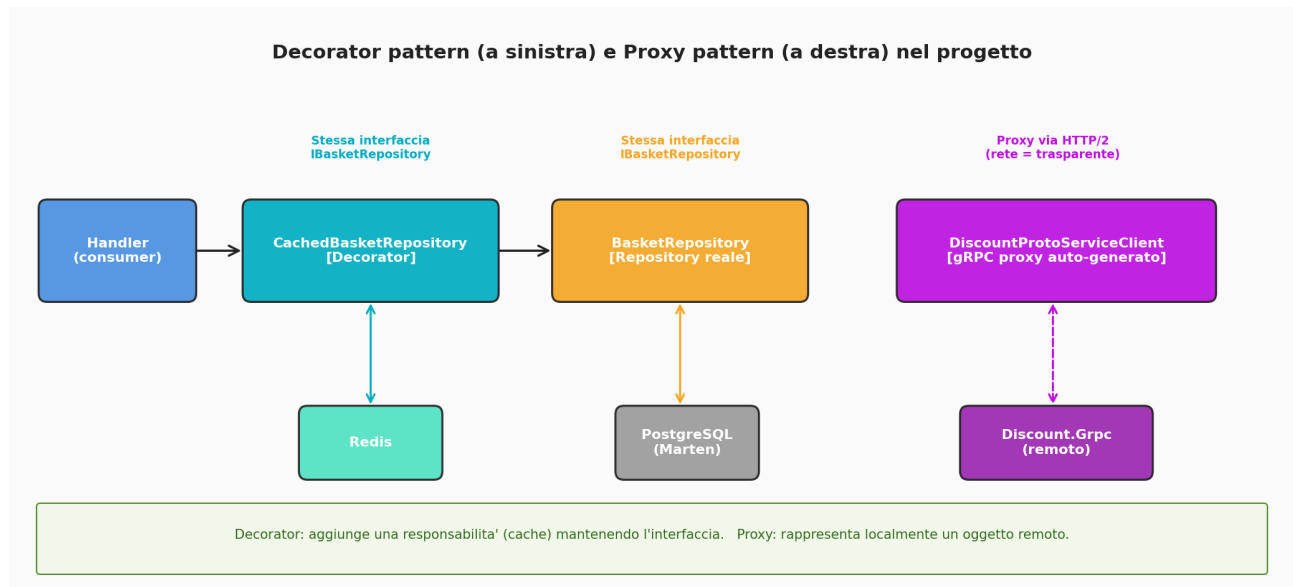
        timer.Stop();
        if (timer.Elapsed.TotalSeconds > 3)
            logger.LogWarning($"[PERF] {typeof(TRequest).Name}...");
        logger.LogInformation($"[END] Handler {typeof(TRequest).Name}");
        return response;
    }
}
```

Aggiungere un nuovo cross-cutting concern (caching, transazioni, auth, telemetry...) significa scrivere una nuova classe che implementa `IPipelineBehavior<, >` e registrarla con `AddOpenBehavior`. Il codice di business non cambia.

17. Decorator + Proxy

Decorator e Proxy sono pattern *strutturalmente quasi identici*: entrambi avvolgono un oggetto che implementa un'interfaccia, esponendo la stessa interfaccia. Cambia l'**intento**:

- **Decorator**: aggiunge una *responsabilit * all'oggetto originale (es. caching, logging, retry).
- **Proxy**: *rappresenta* un altro oggetto (es. uno remoto, uno costoso da creare, uno protetto).



A sinistra Decorator (CachedBasketRepository); a destra Proxy (client gRPC auto-generato).

CachedBasketRepository - Decorator + Proxy insieme

Nel codice del progetto trovi un commento esplicito sulla classe:

```
// CachedBasketRepository.cs
/// <summary>
/// In questa classe implemento sia il proxy pattern che il pattern decorator.
/// Il proxy pattern: la classe CachedBasketRepository funge da proxy per
/// accedere alla repository reale, inoltrando le chiamate alla repository
/// sottostante.
/// Successivamente estenderemo le funzionalita' della classe
/// CachedBasketRepository implementando il pattern decorator.
/// </summary>
public class CachedBasketRepository(
    IBasketRepository basketRepository,
    IDistributedCache cache) : IBasketRepository
{
    public async Task<ShoppingCart> GetBasketAsync(string userName, ...)
    {
        // Decorator: aggiunge il caching prima/dopo
        var cached = await cache.GetStringAsync(userName, ...);
        if (!string.IsNullOrEmpty(cached))
            return JsonSerializer.Deserialize<ShoppingCart>(cached)!;

        // Proxy: inoltra la chiamata al repository reale
        var basket = await basketRepository.GetBasketAsync(userName, ...);
        await cache.SetStringAsync(userName, JsonSerializer.Serialize(basket), ...);
        return basket;
    }
    // ... StoreBasketAsync, DeleteBasketAsync analoghi
}
```

La registrazione magica con Scrutor

In Program.cs del Basket.API trovi:

```
builder.Services.AddScoped<IBasketRepository, BasketRepository>();
builder.Services.Decorate<IBasketRepository, CachedBasketRepository>();
//
//          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
//          fornita dal package Scrutor
```

Decorate dice al DI container: *'quando qualcuno chiede IBasketRepository, creagli un CachedBasketRepository, e iniettagli dentro il BasketRepository reale'*. Aggiungere caching diventa una sola riga di config.

Il client gRPC come Proxy

Quando il Basket chiama `Discount.Grpc`, lo fa tramite una classe auto-generata da `discount.proto`: `DiscountProtoServiceClient`. Per il chiamante questa classe sembra un normale oggetto C#. In realta' ogni metodo:

- Serializza i parametri in Protocol Buffers.
- Invia il pacchetto via HTTP/2 al servizio Discount remoto.
- Aspetta la risposta, la deserializza, la ritorna.

E' la definizione classica di **Remote Proxy**: l'oggetto locale rappresenta un'entita' remota e ne nasconde la complessita' di rete. Tu chiami metodi C# come se fossero locali.

```
// in CheckoutBasketHandler (semplificato)
var coupon = await discountProtoClient
    .GetDiscountAsync(new GetDiscountRequest { ProductName = item.ProductName });
// internamente: serializza, invia su HTTP/2, riceve, deserializza
```

Decorator vs Proxy in tabella

	Decorator	Proxy
Scopo	Aggiungere una responsabilita'	Rappresentare un altro oggetto
L'oggetto avvolto e'...	Lo stesso tipo, ma 'piu' semplice'	Un altro processo / spazio di memoria
Nel progetto	CachedBasketRepository	DiscountProtoServiceClient
Trasparente per il client? Si'		Si'

18. Repository Pattern

Il **Repository Pattern** astrae l'accesso ai dati: l'applicazione lavora con un'interfaccia (`IBasketRepository`, `IOrderingDbContext`) e non sa, ne' deve sapere, se sotto c'e' Postgres, SQL Server, un file JSON o un mock.

Esempio: `IBasketRepository`

```
public interface IBasketRepository
{
    Task<ShoppingCart> GetBasketAsync(string userName, CancellationToken ct = default);
    Task<ShoppingCart> StoreBasketAsync(ShoppingCart basket, CancellationToken ct = default);
    Task<bool> DeleteBasketAsync(string userName, CancellationToken ct = default);
}

// Implementazione 1 (real): Marten + PostgreSQL
public class BasketRepository(IDocumentSession session) : IBasketRepository { ... }

// Implementazione 2 (decorator): aggiunge cache Redis
public class CachedBasketRepository(IBasketRepository inner, IDistributedCache cache)
    : IBasketRepository { ... }
```

Vantaggi nel progetto:

- **Sostituibilita'**: domani potresti passare da Marten a EF Core modificando solo l'implementazione.
- **Componibilita'**: la cache si applica come decorator senza modificare l'interfaccia.
- **Testabilita'**: il test del CommandHandler mocka `IBasketRepository` e basta.

Variante in Ordering: `IOrderingDbContext`

Nel servizio Ordering vedi una variante: invece di un repository per aggregato, c'e' una sola interfaccia `IOrderingDbContext` che espone i `DbSet` rilevanti. E' un Repository di livello piu' alto - sostanzialmente un Unit of Work.

i Repository vs DbContext - quando usare cosa?

Per CRUD semplice (Basket, Catalog) un repository per aggregato e' comodo e leggibile.

Per dominio ricco con piu' aggregati che si toccano (Ordering), esporre direttamente `IOrderingDbContext` evita di moltiplicare le interfacce e supporta in modo naturale le *transazioni multi-aggregato* (un unico `SaveChanges`).

Non c'e' una scelta universale: dipende dalla complessita' del dominio.

19. Aggregate Root, Factory Method, Domain Events (DDD)

Il servizio **Ordering** e' modellato secondo i principi del **Domain-Driven Design**. In una sola classe (Order) trovi *tre* pattern messi insieme: Aggregate Root, Factory Method e Domain Events.

Aggregate Root

Order e' il **punto di ingresso** all'aggregato ordini. Tutto cio' che fa parte dell'ordine (gli OrderItem, gli Address, il Payment) passa solo attraverso Order. Non puoi modificare un OrderItem direttamente: devi chiamare un metodo di Order.

```
public class Order : Aggregate<OrderId>
{
    private readonly List<OrderItem> _orderItems = new();
    public IReadOnlyList<OrderItem> OrderItems => _orderItems.AsReadOnly();
    public CustomerId CustomerId { get; private set; } = default!;
    public OrderName OrderName { get; private set; } = default!;
    public Address ShippingAddress { get; private set; } = default!;
    // ... tutti i setter sono PRIVATE

    public void Add(ProductId productId, int qty, decimal price)
    {
        ArgumentOutOfRangeException.ThrowIfNegativeOrZero(qty);
        ArgumentOutOfRangeException.ThrowIfNegativeOrZero(price);
        _orderItems.Add(new OrderItem(this.Id, productId, qty, price));
    }
}
```

Notare: i setter sono **private**. La collezione e' esposta come IReadOnlyList. Le modifiche passano *solo* dai metodi del dominio. Cosi' le invarianti dell'ordine non possono essere violate.

Factory Method

Non puoi fare `new Order(...)`: il costruttore di Order e' privato. Puoi creare un Order solo tramite il metodo statico Create:

```
public static Order Create(
    OrderId orderId, CustomerId customerId, OrderName orderName,
    Address shippingAddress, Address billingAddress, Payment payment)
{
    var order = new Order
    {
        Id = orderId,
        CustomerId = customerId,
        OrderName = orderName,
        ShippingAddress = shippingAddress,
        BillingAddress = billingAddress,
        Payment = payment,
        Status = OrderStatus.Pending
    };
    order.AddDomainEvent(new OrderCreatedEvent(order));
    return order;
}
```

Lo stesso schema vale per `Address.Of(...)`, `Payment.Of(...)`, `OrderId.Of(...)`: ogni Value Object/aggregato si crea solo tramite un metodo factory. Così il dominio ha sempre il controllo sulle pre-condizioni.

Domain Events

Quando l'`Order.Create` viene chiamato, l'ordine si ricorda di essere stato creato. Lo fa accumulando un evento di dominio in una lista:

```
public abstract class Aggregate<TId> : Entity<TId>, IAggregate<TId>
{
    private readonly List<IDomainEvent> _domainEvents = new();
    public IReadOnlyList<IDomainEvent> DomainEvents => _domainEvents.AsReadOnly();

    public void AddDomainEvent(IDomainEvent domainEvent)
        => _domainEvents.Add(domainEvent);

    public IDomainEvent[] ClearDomainEvents()
    {
        var events = _domainEvents.ToArray();
        _domainEvents.Clear();
        return events;
    }
}
```

L'aggregato *non pubblica* i propri eventi: li accumula. Sara' un altro componente a raccogliarli e dispatcharli al momento giusto - vedi il capitolo successivo sull'**Interceptor**.

! Domain Event vs Integration Event - non confondere

Domain Event (es. `OrderCreatedEvent` del namespace `Ordering.Domain.Events`) e' *interno* al servizio. Viaggia via MediatR all'interno dello stesso processo.

Integration Event (es. `BasketCheckoutEvent` di `BuildingBlocks.Messaging.Events`) e' *esterno*: viaggia via RabbitMQ tra servizi diversi.

I due hanno lo stesso pattern (qualcosa e' successo) ma vivono a livelli diversi - non rompere mai questo confine.

20. Interceptor Pattern (EF Core)

Nel servizio Ordering ci sono due interceptor di EF Core: `AuditableEntityInterceptor` e `DispatchDomainInterceptor`. Si registrano in `DependencyInjection.cs`:

```
services.AddScoped<ISaveChangesInterceptor, AuditableEntityInterceptor>();
services.AddScoped<ISaveChangesInterceptor, DispatchDomainInterceptor>();

services.AddDbContext<OrderingDbContext>((sp, options) =>
{
    options.AddInterceptors(sp.GetServices<ISaveChangesInterceptor>());
    options.UseSqlServer(connectionString);
});
```

DispatchDomainInterceptor: dispatch automatico dei Domain Events

Ricorda: gli aggregati accumulano i Domain Events in una lista interna. L'interceptor li raccoglie e li pubblica via MediatR **nello stesso istante** in cui EF Core sta per salvare le modifiche:

```
public class DispatchDomainInterceptor(IMediator mediator) : SaveChangesInterceptor
{
    public override async ValueTask<InterceptionResult<int>> SavingChangesAsync(
        DbContextEventData eventData, InterceptionResult<int> result,
        CancellationToken ct = default)
    {
        await DispatchDomainEvents(eventData.Context);
        return await base.SavingChangesAsync(eventData, result, ct);
    }

    public async Task DispatchDomainEvents(DbContext? context)
    {
        if (context == null) return;
        var aggregates = context.ChangeTracker.Entries<IAggregate>()
            .Where(a => a.Entity.DomainEvents.Any())
            .Select(a => a.Entity);
        var domainEvents = aggregates.SelectMany(a => a.DomainEvents).ToList();
        aggregates.ToList().ForEach(a => a.ClearDomainEvents());
        foreach (var domainEvent in domainEvents)
            await mediator.Publish(domainEvent);
    }
}
```

Cosa abbiamo guadagnato:

- Il dominio resta puro: `Order` non sa nulla di MediatR, di EF Core, di nessuna infrastruttura.
- La pubblicazione degli eventi avviene **nella stessa transazione** del salvataggio: o tutto o niente.
- Il Command Handler scrive solo la logica di business (`Order.Create`, `SaveChanges`): nessuna riga dedicata alla pubblicazione di eventi.

AuditableEntityInterceptor: timestamp gratis

L'altro interceptor popola automaticamente i campi `CreatedAt`, `CreatedBy`, `LastModified`, `LastModifiedBy` su tutte le entita' che ereditano da `Entity`. Lo sviluppatore non se ne preoccupa

- i campi 'audit' si compilano da soli.

i Pattern Interceptor

Concettualmente identico al **Pipeline Pattern** di MediatR e ai **Middleware** di ASP.NET: si inserisce codice trasversale prima/dopo un'operazione (qui il SaveChanges) senza che il chiamante debba saperlo. Cambia solo il punto di aggancio.

21. Riepilogo e punti chiave da ricordare

La regola d'oro

Il publisher non parla mai direttamente al consumer. Parla solo con il broker. E il consumer non sa chi ha generato il messaggio: ascolta solo il broker. Questa indipendenza e' il motivo per cui i microservizi possono evolvere separatamente.

Le 5 cose che non devi piu' dimenticare

- **1. Asincrono = scrivere, non aspettare.** Publish ritorna subito; il lavoro vero accade dopo, nel consumer.
- **2. RabbitMQ e' fuori dalla tua app.** E' un container Docker. Tu ci parli via il protocollo AMQP - ma usando MassTransit scrivi solo classi C#.
- **3. MassTransit traduce.** Trasforma `Publish(BasketCheckoutEvent)` in comandi AMQP, e i comandi AMQP che arrivano dalla queue in chiamate al tuo `Consume(...)`.
- **4. Una queue per consumer.** Se domani aggiungi un `NotificationService` che vuole sapere quando avviene un checkout, basta creare un altro `IConsumer<BasketCheckoutEvent>`: MassTransit creera' una sua queue e riceverà una copia del messaggio.
- **5. Il broker resiste alle tempeste.** Se Ordering crolla, Basket continua a pubblicare, RabbitMQ accumula i messaggi. Quando Ordering torna su, recupera tutto cio' che ha perso.

Una mappa mentale finale

= In una frase

Il **Basket** butta una lettera nella **cassetta postale** RabbitMQ ogni volta che un cliente chiude un ordine, e **Ordering** passa di tanto in tanto a leggerla per creare l'ordine vero. **MassTransit** e' il fattore che si occupa di buste, francobolli e di consegnare a domicilio - tu scrivi solo il messaggio.

Fine del documento - Buon coding, Hassan!